bench4JSBACH Documentation

Release 0.0.1

Deike Kleberg

Oct 06, 2016

Contents

1 Introduction **1**

2 User guide for benchmarking **3**

3 User guide for developing `diagnostic` `field` s **10**

4 Code documentation **12**

Bibliography **22**

Python Module Index **23**

Index **24**

Contents:

Introduction

Main idea

With **bench4jsbach** we started developing a tool for the scientists of the land department in the Max Planck Institute for Meteorology to make the comparison of JSBACH model simulation output data *[ReJSBA]* with one another and with data products straight forward and user friendly. In the first instance the tool shall serve for the benchmarking of land models, it has to be able to quantify the deviation of a simulation from reference experiments in the most crucial and/or influential parameters out-of-the-box. This will help developing models as well as managing a large number of them. It is desirable to cover most research areas of the land department and to keep the tool well adapted to the local technical conditions at the Max Planck Institute for Meteorology.

In order to have a living, dynamical tool, the possibility for individual extension is provided to the user from the beginning. This will help the benchmarking process to retain its validity but also help the scientists to handle future questions bringing new values into focus. It may foster the communication between the groups as individual developments can easily be made usable for the whole department and thus serve as a basis for a fruitfuly discussion on the most significant diagnostic for the generally applicable benchmarking in the department.

Concepts

A guiding idea for **bench4jsbach** is the separation of issues that strongly depend on the origin of the climate data from issues directly related with their comparison. It is intended to solve the origin dependend tasks 'under the hood' of the tool so that a scientist can focus on the diagnostic and benchmarking methods.

Experiments

We use the generic term `experiment` independent from the data origin being an earth system model simulation or an observation. Origin dependend processing like locating and extracting necessary raw data from different host, filesystem and out of datafile but also data interpolation are gathered under the name.

You may assign the role of a reference or that of a candidate to an `experiment`, in a comparison at least one instance of each is needed. It is possible to have multiple `experiment` s and also a single instance can take both roles in one **bench4jsbach** run. The main difference of the roles is the comparison being made and the report being written from the perspective of the candidate `experiment` s. Depending on the evalution method each candidate can be compared with single references or with all references at once.

Diagnostic fields

The term `diagnostic field` clusters processing for the comparison of `experiment` s with a defined set of variables featuring a particular scientific question. It should contain at least one diagnostic and one benchmarking method. In this context the term benchmarking refers to a method that condenses the deviation of two experiments to one number, i.e. yields a global and time average value.

Context

The following diagram shows **bench4jsbach** in context with other actors (users, software, ...).

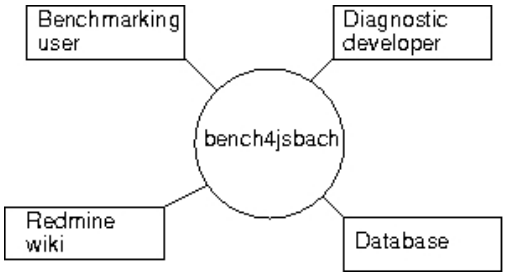


Fig. 1: **Figure:** Context diagram of **bench4jsbach**

Benchmarking user

- scientist managing many models
- model developer making still-in-line-check

Diagnostic developer dedicated scientist integrating a `diagnostic field`

Database database for reference `experiment` s

Redmine wiki communication platform for **bench4jsbach**

Status

This documentation is based on a not yet tagged version (*0.0.1*) of **bench4jsbach**, a vertical prototype. The basic operating concept, the program structure and the workflow is implemented. The tool handles `experiment` s of type MPIESM1 (Max Planck Institute for Meteorology Earth System Model, *[GiESM1]*) and a `diagnostic field` analysing albedo.

User guide for benchmarking

This chapter shall enable you to use **bench4jsbach** for the comparison of `experiment` s with predefined `diagnostic field` s.

Prerequisites

bench4jsbach is developed for UNIX and LINUX operating systems. It is written in object-oriented Python and works with version 2.7. The packages NumPy and NetCDF4 and the 2D plotting library matplotlib are used which are contained in most scientific Python distributions like Anaconda, Miniconda or Canopy. Furthermore **bench4jsbach** makes use of the benchmarking tool pyCMBS and the climate data command line operators CDO, both inhouse-developments of the Max-Planck-Institute for Meteorology.

We use Anaconda and can highly recommand it, as it simplifies package management and deployment, see http://conda.pydata.org/docs/_downloads/conda-cheatsheet.pdf. From here on it is refered to in the installation process.

Make sure that your system uses Python from `opt/anaconda/bin` with `opt` being your Anaconda base directory. If necessary, download and install Anaconda, you can get it from http://docs.continuum.io/anaconda/install#linux-install, and put `opt/anaconda/bin` to your system PATH. Then update and install the necessary Python packages, ..

```
>>> conda update conda
>>> conda update anaconda
>>> conda install netcdf4
>>> conda install -c https://conda.binstar.org/scitools cartopy
>>> conda install geos basemap
>>> conda install gdal
```

.. CDO ..

```
>>> pip install cdo
```

.. and pyCMBS

```
>>> cd opt
>>> git clone https://github.com/pygeo/pycmbs
>>> cd pycmbs
>>> env OMP_NUM_THREADS=1 python setup.py install
```

Get bench4jsbach

Change to the root directory of your local JSBACH repository git@git.mpimet.mpg.de:jsbach.git, that is the one containing the `.git` directory. If you do not have the repository get it with

```
>>> git clone git@git.mpimet.mpg.de:jsbach.git
```

Therefore you need passwordless ssh and the access permission for the git server (contact reiner.schnur@mpimet.mpg.de). Switch to the git branch `jsbach − dkl`

```
>>> git checkout jsbach-dkl
```

You will also need passwordless ssh for the servers from which you want to read experiment data.

Being in the git branch `jsbach − dkl` you find the files for the tool under `git/jsbach/tools/bench4jsbach` with `git/jsbach` as your git root directory. The structure of the **bench4jsbach** base directory `bench4jsbach` you find initially is shown below.

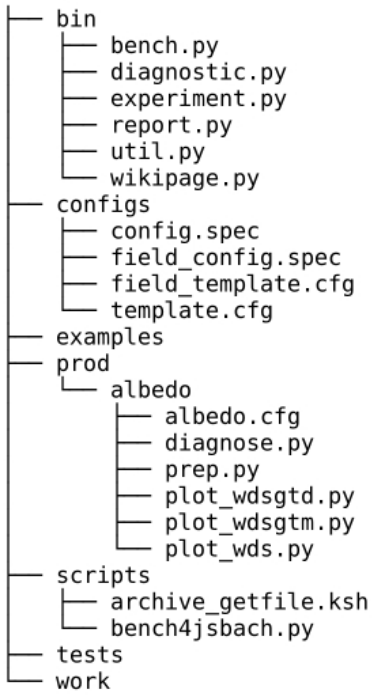


Fig. 2: **Figure:** initial structure of `git/jsbach/tools/bench4jsbach`

The following list describes the individual directories in `bench4jsbach`.

bin core source files

configs specification and template files for the run configuration

doc Sphinx documentation source files and Makefile

prod scripts and configurations for particular diagnostic fields

scripts essential parts of **bench4jsbach** e.g. run scripts

tests different testing environments

work currently hardcoded working directory

Prepare a run

A **bench4jsbach** run consists of two parts, the initial startup run which is done once to set up the environment and create the directory structure and the main run, fetching expriment data, analysing, making plots and writing a report. Both parts are controled by the run script `bench4jsbach/scripts/bench4jsbach.py`.

Initial run

A **bench4jsbach** run needs a name as it writes data to your local machine that has to be addressed. First choose this run name (e.g. `myb4jrun`) and change to the base directory `bench4jsbach`. Call the run script `bench4jsbach.py` with your run name as command line argument.

```
>>> cd git/jsbach/tools/bench4jsbach
>>> ./scripts/bench4jsbach.py myb4jrun
```

While running, the tool writes a welcome and a final success/error message to your display. In addition the internal debugger prints to standard output in the default setting.

```
*****************************************
* BENCH4JSBACH:
* Creating environment and configfile for new bench4jsbach object 'myb4jrun'.
*****************************************

### DKL_LOGGER - DEBUG -  2016/10/04 - 18:12:08 - MODULE bench - FUNCTION __init__ - LINE 143 :
  # Starting.
###

### DKL_LOGGER - DEBUG -  2016/10/04 - 18:12:08 - MODULE bench - FUNCTION _writeconfig__init__ - LINE
  # Starting.
###

### DKL_LOGGER - DEBUG -  2016/10/04 - 18:12:08 - MODULE bench - FUNCTION _writeconfig__init__ - LINE
  # Exiting.
###

### DKL_LOGGER - DEBUG -  2016/10/04 - 18:12:08 - MODULE bench - FUNCTION __init__ - LINE 147 :
  # Exiting.
###

*****************************************
* BENCH4JSBACH:
* Created environment
*         '/home/git/jsbach/tools/bench4jsbach/work/myb4jrun'
* and the configuration file
*         '/home/git/jsbach/tools/bench4jsbach/work/myb4jrun/configs/myb4jrun.cfg'.
* Please edit this file.
*****************************************
```

Listing: Standard output after the initial run with argument `myb4jrun`

When the startup run has finished successfully, the directory structure for `myb4jrun` is created (as shown in the figure below) and an editable, default configuration file `myb4jrun.cfg` is written in `myb4jrun/configs`, also shown further down.



Fig. 3: **Figure:** Initial structure of `bench4jsbach/work/myb4jrun`

Configuration file

The generated file is a form for the main configuration of your **bench4jsbach** run and has to be adapted to the planned experiment comparison. It is formatted to be read by a Python config file parser after your accomplishment. A config file consists of key value pairs and sections nested to any depth. For the sections square brackets are used, indentation clearifies the nesting level. You can write comments inline prepended by a hash symbol "#".

```
key-1=value-1
key-2=value-2
[section-1]
   key-1=value-1
   [[subsection-1]]
      key-1=value-1
```

Listing: Example configuration

After generation the file `myb4jrun.cfg` contains the structure of information that the configuration for a minimum **bench4jsbach** run with two `experiment` s (see *Experiments*) and one `diagnostic field` (see *Diagnostic fields*) has to provide.

In the file `myb4jrun.cfg`, which is displayed below, the mandatory fields to be filled in are marked with angle brackets (<,>) and located in the second part of the file, below the comment line "# PLEASE EDIT ...". The capital text within is meant to be explanatory and the whole string including the brackets has to be exchanged against your value for the respective item.

```
################################################################
#
# configuration file for bench4jsbach object 'myb4jrun'
#
# automatically generated on 2016-09-22
#
################################################################

### stamp like meta data, do not edit

bench_name=myb4jrun
author=deike
git_repository=origin(fetch):git@git.mpimet.mpg.de:jsbach.git,origin(push):git@git.mpimet.mpg.de:jsba
git_branch=jsbach-dkl
git_sha=167c9e4d3h5df92c0e8e8234878698b5956b35d1
git_status=modified
redmine_url=https://code.zmaw.de
redmine_project=jsbach-bench
redmine_wikipage=Bench4jsbach_simulations
host_name=scalar
```

```
host_ipaddress=127.0.1.1
b4j_base=/home/deike/git/jsbach/tools/bench4jsbach
timestamp=2016-09-22 11:26:48.446387

################################################################
#
# PLEASE EDIT FROM HERE ON DOWNWARD
#
################################################################

### global meta data

redmine_user=<REDMINE_USER>
redmine_key=<REDMINE_KEY>
institute="Max-Planck-Institute for Meteorology"
department="Land Department"
reporting=<BENCH_IS_REPORTING>

### experiments in diagnostic run, candidates and references

[experiments]
    [[<MODEL_1_ID>]]
        name_orig=<MODEL_1_NAME_ORIGINAL>
        type=<MODEL_1_TYPE>
        path=<MODEL_1_PATH>
        host=<MODEL_1_HOST>
        iscan=<MODEL_1_IS_CANDIDATE>
        isref=<MODEL_1_IS_REFERENCE>
        startyear=<MODEL_1_STARTYEAR_FOR_THIS_RUN>
        endyear=<MODEL_1_ENDYEAR_FOR_THIS_RUN>
        intitalyear=<MODEL_1_INITIALYEAR_OVERALL>
        finalyear=<MODEL_1_FINALYEAR_OVERALL>
    [[<MODEL_2_ID>]]
        name_orig=<MODEL_2_NAME>
        type=<MODEL_2_TYPE>
        path=<MODEL_2_PATH>
        host=<MODEL_2_HOST>
        iscan=<MODEL_2_IS_CANDIDATE>
        isref=<MODEL_2_IS_REFERENCE>
        startyear=<MODEL_2_STARTYEAR_FOR_THIS_RUN>
        endyear=<MODEL_2_ENDYEAR_FOR_THIS_RUN>
        intitalyear=<MODEL_2_INITIALYEAR_OVERALL>
        finalyear=<MODEL_2_FINALYEAR_OVERALL>

### diagnostic fields

[fields]
    [[<FIELD_1_NAME>]]
        config_filename=<FIELD_1_CONFIGURATION_FILENAME>
```

Listing: Main configuration file `myb4jrun.cfg`

In the following the values to fill in are summarized.

Top level config

The redmine access is necessary if the **bench4jsbach** run has to read reference `experiment` metadata from the wikipage.

key		value	
name	description	datatype	default
redmine_user	redmine user login name	string	None
redmine_key	redmine "Api access key"	string	None
institute	users' institute	string	None
department	users' department	string	None
reporting	reporting switch	booolean	True

`experiment` section

For each `experiment` in the run there has to be a subsection that is labeled with a run internal unique name.

Single `experiment` subsection level:

key		value	
name	description	datatype	default
name_orig	original identifying infix e.g. "vga0187"	string	-
type	`experiment` type	option (CMIP5,MPIESM1)	MPIESM1
path	• path of the data • if reference from wikipage	string	None
host	• server hosting the data • if reference from wikipage	string	None
iscan	candidate role	boolean	-
isref	reference role	boolean	-
startyear	startyear in **bench4jsbach** run	string	-
endyear	endyear in **bench4jsbach** run	string	-
initialyear	initial year of `experiment`	string	None
finalyear	final year of `experiment`	string	None

`diagnostic field` section

For each diagnostic field in the run has to be a subsection that is labeled with a run internal unique name.

Single diagnostic field subsection level in the main configuration file for a **bench4jsbach** run:

key		value	
name	description	datatype	default
config_filename	one of two options: • field config filename in bench4jsbach/prod/configs • path to field config file on local machine	string	-

Files in context with the available diagnostic fields are stored in `bench4jsbach/prod`, the configuration files beneath in the directory `configs`. A list of available diagnostic fields can be found below (<LINK>).

Run example `myb4jrun`

When you have replaced all placeholders "<XXX>" with the appropriate values the bottom part of your configuration file should look in principle like the example below.

```
### global meta data

redmine_user='deike.kleberg'
redmine_key='XXX'
institute="Max-Planck-Institute for Meteorology"
department="Land Department"
reporting=True

### experiments in diagnostic run, candidates and references

[experiments]
    [[tra0163-0]]
        name_orig=tra0163
        type=MPIESM1
        iscan=False
        isref=True
        startyear=3300
        endyear=3329
    [[tra0163-1]]
        name_orig=tra0163
        type=MPIESM1
        iscan=True
        isref=False
        startyear=3970
        endyear=3999

### diagnostic fields

[fields]
    [[albedo]]
        config_filename=b4j_albedo.cfg
```

Listing: Extract of example configuration file `myb4jrun.cfg`

The run can now be started.

```
>>> ./scripts/bench4jsbach.py work/myb4jrun/myb4jrun.cfg
```

The CPU time maybe about one minute but, as it depends on the duration of the data fetching process, also more.

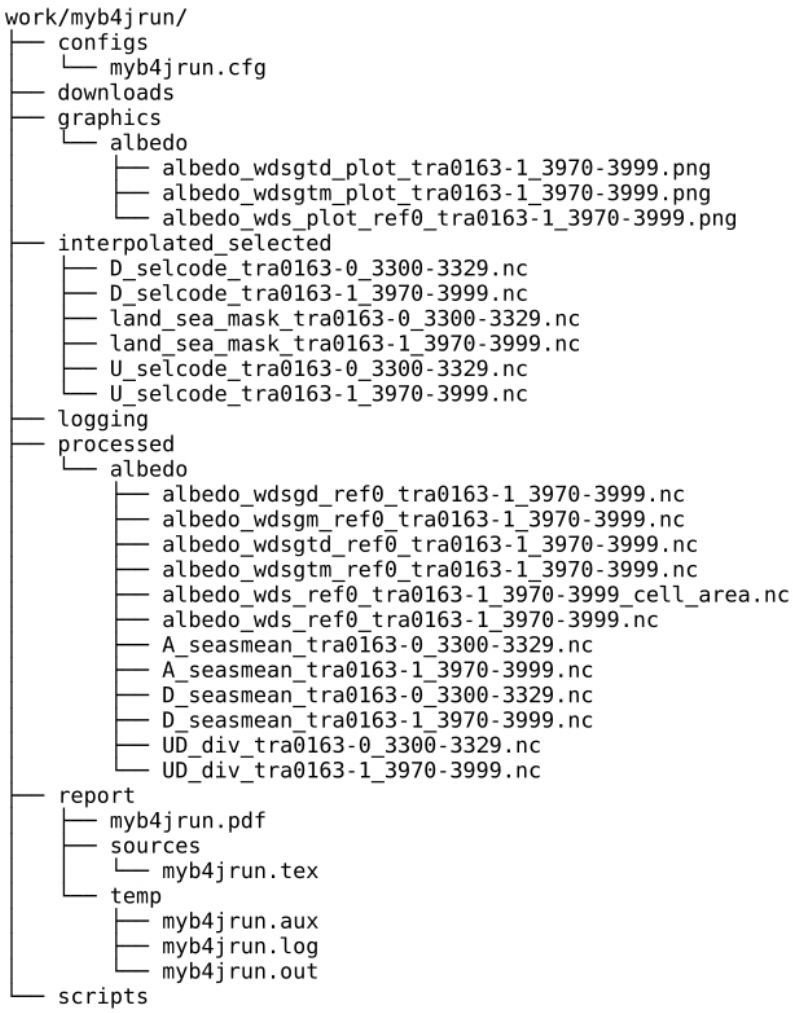


Fig. 4: **Figure:** After the run, structure of `bench4jsbach/work/myb4jrun`

The generated report `myb4jrun.pdf` can be found in `myb4jrun/report`, LaTeX source and temporary files are kept in subdirectories, the graphics included in the report are in a parallel directory `myb4jrun/graphics`. If you want to have a closer look on the diagnostics made, you find the results of the calculations and also the intermediate, preprocessed and downloaded data in the directories `downloads`, `interpolated_selected` respectively `processed`.

References

User guide for developing `diagnostic field` s

In the following you learn how to develop a `diagnostic field` and to integrate it in a **bench4jsbach** run.

A diagnostic for experiment comparison from external sources can be included as `diagnostic field` into a **bench4jsbach** run. Therefore it needs scripts for the processing chain (preprocessing, analysing, plotting), a configuration file and descriptive textblocks for the report. A script may be written in any programming language that is supported by your platform. It has to be layed out for one candidate and one reference so that it can be called in loops over multiple experiments. The scripts work on local data which have been selected and interpolated from the experiment output as indicated by the user and the experiment type. They interface of each script has to be declared in

a respective section of the `diagnostic field` s' configuration file. The information to provide include input and output filenames and possibly hints on format, grid, spatial resolution, tiling and global fragmentation and time resolution and period.

Configuration

The configuration for a `diagnostic field` defines a processing chain that is interconnected by I/O dependencies. For the time being the processes have a binding order: 1. `fetch`, 2. `interpolate`, 3. `preprocess`, 4. `diagnose`, 5. `plot`. **bench4jsbach** works internally with an indexing system in which files are addressed by unique infixes, f.e. `FILE − INFIX`. For each file that shall be written by a process the appropriate section has to hold a subsection [[`FILE − INFIX`]] wich contains the detailed configuration. This infix will also be used as substring for the respective filename. If the file has to be read by a subsequent process the infix is used in the respective configuration as value for its `input` item. For illustration the listing below shows a detail from the configuration file for the `diagnostic field Albedo`. Clicking on the graphic opens another picture, showing how the I/O dependencies are taken into account.

```
[fetch]
    [[D_select]]
        [[[MPIESM1]]]
            code=21
            submodel=jsbach
            stream=jsbach
    [[U_select]]
        [[[MPIESM1]]]
            code=22
            submodel=jsbach
            stream=jsbach

[interpolation]

[preprocessing]
    exec=prep.ex
    input=D_select,U_select,
    [[UD_div]]
    [[A_seasmean]]
    [[D_seasmean]]
                              ...
```

```
...
[diagnostic]
    exec=diagnostic.ex
    input=A_seasmean,D_seasmean,
    [[alb_wds]]
    [[alb_wdsgm]]
    [[alb_wdsgtm]]
        metric=True

[plotting]
    suffix=png
    [[wds_plot]]
       exec=plot_wds.ex
       input=alb_wds,
    [[wdsgtm_plot]]
       exec=plot_wdsgtm.ex
       input=alb_wdsgm,alb_wdsgtm,

[report]
```

Listing: albedo.cfg, example

As can be seen in detail in the *User guide for benchmarking* a `diagnostic field` has to be introduced by a labeled subsection in the diag-f of the main configuration file providing in particular the filename respectively the whole local path for the specialized configuration.

Fetch section

Each variable that has to be read from the experiment's dataset is represented by a subsection labeled with a run internal unique name. The data fetched and selected from the experiment output will be written to the subdirectory `interpolated_selected` in `work`, f.e. `work/myb4jrun/interpolated_selected` with a filename containing this name as infix. The single variable subsection level has subsubsections for each experiment type holding specialized configuration.

Single variable subsection:

key		value	
name	description	datatype	default
longname	long descriptive name	string	None
unit	physical unit	string	-

`MPIESM1` subsubsection:

key		value	
name	description	datatype	default
name	variable name in datafile	string	-
code	variable code number in file	string	-
submodel	hosting submodel in coupled model	string	None
stream	stream, hosting datafile	string	None

Processing sections, `preprocessing`, `diagnose`, `plotting`:

key		value	
name	description	datatype	default
suffix	format for output files	option (nc)	nc
exec	executable script	string	-
input	data needed from precursor	string	-

Specialized configuration for the processing section `plotting`:

key		value	
name	description	datatype	default
suffix	format for output files	option (png)	png
nref	members in comparison	option (single, multiple)	single
plot_title	title for the plot	string	None

Processing scripts

For integrated `diagnostic field` s the processing scripts can be found under `prod/scripts`. If you want to use your own `diagnostic field` you can provide the paths for the executable scripts under the item `exec` in the respective process' subsection of the configuration file.

As this is work in progress it has to be reckoned with major changes while more `diagnostic field` s and other experiment types are integrated.

Code documentation

This page provides an auto-generated summary of bench4jsbach's code documentation.

Bench

The module contains bench related functionality and data.

Functions

bench.get_git_origin()

Continued on next page

Table 1 – continued from previous page

bench.get_git_branch()
bench.get_git_sha()
bench.get_git_status()
bench.get_ip_address()

bench.get_git_origin

bench.**get_git_origin**()

bench.get_git_branch

bench.**get_git_branch**()

bench.get_git_sha

bench.**get_git_sha**()

bench.get_git_status

bench.**get_git_status**()

bench.get_ip_address

bench.**get_ip_address**()

Creating a bench object

bench.Bench_Object(config_arg, **kwargs)	This object coordinates the bench4jsbach run.

bench.Bench_Object

class bench.**Bench_Object**(*config_arg*, ***kwargs*)

Bases: *diagnostics.Diagnostics*, *report.Report*

This object coordinates the bench4jsbach run.

__init__(*config_arg*, ***kwargs*)

This object coordinates the bench4jsbach run.

Args:

config_arg (str): If config_arg is a local path to an existing bench4jsbach configuration file a run is started. Else config_arg is taken as the name for a new bench4jsbach environment, this is initialized and the configuration file is generated.

Methods

__init__(config_arg, **kwargs)	This object coordinates the bench4jsbach run.
diagnostic_generic(fieldname)	
get_b4j_path(filename, b4j_filetype)	
init_diagnostics()	This method is used to initialize the diagnostic fields.
report_diagnostic()	
write_description()	
write_report()	
write_summary()	

Attributes

TeXlib_dirtree
TeXlib_forest
bench_configs_path
bench_name
bench_path
config
configs_path
diagnostic_fields
download_path
experiments
graphics_path
interpolated_selected_path
logfile_path
logger
logger_name
logging_path
n_cans
n_refs
path_template
processed_path
report_path
reporting
scripts_path
wikipage

Diagnostics

diagnostics	The module contains diagnostic related classes, functionality and data.

diagnostics

The module contains diagnostic related classes, functionality and data.

Functions

find_files(pattern, path[, single, raise_e])	
get_largest_common_yearlist(filelist)	
get_seasonal_period_2d_coords(xvals, yvals, ...)	
get_vardes_from_codes(exp_name, code, path)	
print_item_or_list_rec(something, indent)	
subprocessing(cmd[, exit_on_error, ...])	
teXify(string)	
touch(filename)	

Classes

CMIP5_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations done for CMIP5.
Cdo([returnCdf, returnNoneOnError, ...])	
Diagnostics(**kwargs)	
MPIESM1_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations with MPIESM1.
OrderedDict(*args, **kwds)	Dictionary that remembers insertion order
Set([iterable])	Mutable set class.
datetime(year, month, day[, hour[, minute[, ...)	The year, month and day arguments are required.

Creating a diagnostics

diagnostics.Diagnostics(**kwargs)	

diagnostics.Diagnostics

class diagnostics.**Diagnostics**(***kwargs*)

Bases: object

__init__(***kwargs*)

Methods

__init__(**kwargs)	
diagnostic_generic(fieldname)	

Attributes

path_template

Experiment

experiment	Module contains experiment related classes, methods and metadata.

experiment

Module contains experiment related classes, methods and metadata.

Functions

find_files(pattern, path[, single, raise_e])	Function to find files with a name containing a given infix.
getdomain()	
gethostname()	
print_none(item)	
subprocessing(cmd[, exit_on_error, ...])	

Classes

CMIP5_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations done for CMIP5.
Cdo([returnCdf, returnNoneOnError, ...])	
Dataset	A netCDF *netCDF4.Dataset* is a collection of dimensions, groups, variables and att
Experiment(b4j_obj, **kwargs)	Base class representing methods and metadata of an experiment.
MPIESM1_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations with MPIESM1.

Functions

experiment.find_files(pattern, path[, ...])	Function to find files with a name containing a given infix.

experiment.find_files

experiment.**find_files**(*pattern*, *path*, *single=True*, *raise_e=False*)

Function to find files with a name containing a given infix.

Creating an experiment

experiment.Experiment(b4j_obj, **kwargs)	Base class representing methods and metadata of an experiment.
experiment.CMIP5_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations done for CMIP5.
experiment.MPIESM1_Experiment(b4j_obj, **kwargs)	Derived class specialized on simulations with MPIESM1.

experiment.Experiment

class experiment.**Experiment**(*b4j_obj*, ***kwargs*)

Bases: object

Base class representing methods and metadata of an experiment.

Specialized classes for model simulations and observational datasets are derived from it.

__init__(*b4j_obj*, ***kwargs*)

Objects are initialized giving their attributes.

In a bench4jsbach run this is done by the runskript reading a configuration file.

Methods

__init__(b4j_obj, **kwargs)	Objects are initialized giving their attributes.
accumulate_streams(orig_dirpath, ...)	
fetch_archived_streams(fetchfile_list)	
fetch_land_sea_mask(origin_path, ...)	
fetch_outdata()	Fetch outdata.
remotehost_fetching(remote_path, target_path)	
remotehost_processing(cmd)	
teX_repr()	
teX_short_repr([shortname])	

Attributes

B4J_atmres	
B4J_canno	
B4J_endyear	
B4J_etype	
B4J_finalyear	
B4J_host	
B4J_id	
B4J_index	
B4J_initialyear	
B4J_iscan	
B4J_isref	
B4J_landseamask	
B4J_name	
B4J_name_orig	
B4J_obj	
B4J_oceres	
B4J_path	this is a comment on B4J_path
B4J_refno	
B4J_startyear	

experiment.CMIP5_Experiment

class experiment.**CMIP5_Experiment**(*b4j_obj*, ***kwargs*)

Bases: *experiment.Experiment*

Derived class specialized on simulations done for CMIP5.

__init__(*b4j_obj*, ***kwargs*)

Methods

__init__(b4j_obj, **kwargs)	
accumulate_streams(orig_dirpath, ...)	
fetch_archived_streams(fetchfile_list)	

Continued on next page

Table 18 – continued from previous page

fetch_land_sea_mask(origin_path, ...)	
fetch_outdata()	Fetch outdata.
remotehost_fetching(remote_path, target_path)	
remotehost_processing(cmd)	
teX_repr()	
teX_short_repr([shortname])	

Attributes

B4J_atmres
B4J_canno
B4J_endyear
B4J_etype
B4J_finalyear
B4J_host
B4J_id
B4J_index
B4J_initialyear
B4J_iscan
B4J_isref
B4J_landseamask
B4J_name
B4J_name_orig
B4J_obj
B4J_oceres
B4J_path
B4J_refno
B4J_startyear
suffix

experiment.MPIESM1_Experiment

class experiment.**MPIESM1_Experiment**(*b4j_obj*, ***kwargs*)

Bases: *experiment.Experiment*

Derived class specialized on simulations with MPIESM1.

__init__(*b4j_obj*, ***kwargs*)

Methods

__init__(b4j_obj, **kwargs)	
accumulate_streams(orig_dirpath, ...)	
fetch_archived_streams(fetchfile_list)	
fetch_land_sea_mask(origin_path, ...)	
fetch_outdata()	Fetch outdata.
remotehost_fetching(remote_path, target_path)	
remotehost_processing(cmd)	

Continued on next page

Table 20 – continued from previous page

teX_repr()
teX_short_repr([shortname])

Attributes

B4J_atmres
B4J_canno
B4J_endyear
B4J_etype
B4J_finalyear
B4J_host
B4J_id
B4J_index
B4J_initialyear
B4J_iscan
B4J_isref
B4J_landseamask
B4J_name
B4J_name_orig
B4J_obj
B4J_oceres
B4J_path
B4J_refno
B4J_startyear
codesfiles_dict
outfiles_dict
pool_dict

Report

report	The module contains report related classes, functionality and data.

report

The module contains report related classes, functionality and data.

Functions

get_filesystem_tree_dirtree(rootDir[, onlydirs])
get_filesystem_tree_tikz(rootDir[, onlydirs])
includegraphics_template(path[, width, hspace])
lib_avail(libname)
print_item_or_list_rec(something, indent)
print_none(item)
subprocessing(cmd[, exit_on_error, ...])
teXify(string)

Classes

Dataset	A netCDF *netCDF4.Dataset* is a collection of dimensions, groups, variables and attributes.
OrderedDict(*args, **kwds)	Dictionary that remembers insertion order
Report()	

Creating a report

report.Report()

report.Report

class report.**Report**

Bases: object

__init__()

Methods

__init__()
report_diagnostic()
write_description()
write_report()
write_summary()

Attributes

TeXlib_dirtree
TeXlib_forest
diagnostic_fields

Wikipage

wikipage	The module contains redmine related classes, functionality and data.

wikipage

The module contains redmine related classes, functionality and data.

Classes

Bench4JSBach_Wikipage(redmine_user, redmine_key)	
Redmine(url, **kwargs)	An entry point for all requests

Creating a wikipage

wikipage.Bench4JSBach_Wikipage(redmine_user, ...)	

wikipage.Bench4JSBach_Wikipage

class wikipage.**Bench4JSBach_Wikipage**(*redmine_user*, *redmine_key*)

Bases: object

__init__(*redmine_user*, *redmine_key*)

Methods

__init__(redmine_user, redmine_key)	
fetch_simulation_row(experiment_role, ...)	
fetch_simulation_row_split_period(...)	
test()	
upload_file(filepath)	
write_b4j_row()	

Attributes

codezmaw_url
project_name
redmine
redmine_project
redmine_wikipage
wikipage_name

Util

util	Module contains functions and constants of general nature.

util

Module contains functions and constants of general nature.

Functions

get_filesystem_tree_dirtree(rootDir[, onlydirs])
get_filesystem_tree_tikz(rootDir[, onlydirs])
getdomain()
gethostname()
pprint_class_member_methods(modulename, ...)

Continued on next page

Table 34 – continued from previous page

print_item_or_list_rec(something, indent)
print_none(item)
setup_logger(*args, **kwargs)
str2bool(s)
subprocessing(cmd[, exit_on_error, ...])
touch(filename)

Classes

WrappedFixedIndentingLog([fmt, datefmt, ...])

References

[GiESM1] Giorgetta, M., et al.: Climate and carbon cycle changes from 1850 to 2100 in MPI-ESM simulations for the coupled model intercomparison project phase 5. Journal of Advances in Modeling Earth Systems, 5, 572-597 , 2013, doi:10.1002/jame.20038

[ReJSBA] Reick, C., et al.: Representation of natural and anthropogenic land cover change in MPI-ESM. Journal of Advances in Modeling Earth Systems, 5, 459-482 , 2013, doi:10.1002/jame.20022

Python Module Index

d

diagnostics, 14

e

experiment, 16

r

report, 19

u

util, 21

w

wikipage, 20

Index

Symbols

__init__() (bench.Bench_Object method), 13
__init__() (diagnostics.Diagnostics method), 15
__init__() (experiment.CMIP5_Experiment method), 17
__init__() (experiment.Experiment method), 16
__init__() (experiment.MPIESM1_Experiment method), 18
__init__() (report.Report method), 20
__init__() (wikipage.Bench4JSBach_Wikipage method), 21

B

Bench4JSBach_Wikipage (class in wikipage), 21
Bench_Object (class in bench), 13

C

CMIP5_Experiment (class in experiment), 17

D

Diagnostics (class in diagnostics), 15
diagnostics (module), 14

E

Experiment (class in experiment), 16
experiment (module), 16

F

find_files() (in module experiment), 16

G

get_git_branch() (in module bench), 13
get_git_origin() (in module bench), 13
get_git_sha() (in module bench), 13
get_git_status() (in module bench), 13
get_ip_address() (in module bench), 13

M

MPIESM1_Experiment (class in experiment), 18

R

Report (class in report), 20
report (module), 19

U

util (module), 21

W

wikipage (module), 20